Lab 13 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of valkuil, en een voorbeeld dat je aan de terminal kunt nagaan.

Vraag 1 — `host not found in upstream "api"`

Antwoord. In lab 07 gaf de DNS van het Podman-netwerk elke **container** een naam. In het cluster heeft een Pod **geen** DNS-naam: alleen **Services** hebben er een. nginx resolvet `api` bij het opstarten (lab 07), vindt niets, en eindigt met een fout; de kubelet herstart hem, vandaar `CrashLoopBackOff`. Je moet het Service `api` aanmaken (selector `app: api`, poort 8080): CoreDNS maakt dan de vermelding aan, en bij de volgende herstart van de backoff start nginx — `web` herstart vanzelf omdat `restartPolicy: Always` eindeloos blijft proberen.

Waarom. Het benoemen is een verantwoordelijkheid van het Service, niet van de Pod: Pods zijn te vluchtig om een naam te verdienen.

Nuance. In een volledig manifest pas je het Service **vóór** of **samen met** het Deployment van de frontend toe; de volgorde in eenzelfde `apply -f map/` is alfabetisch, vandaar de gewoonte om alles in één bestand te zetten of de backoff zijn werk te laten doen.

Voorbeeld.

```
kubectl apply -f service-api.yaml && kubectl rollout status deployment/web --timeout=120s   # successfully rolled out
```

Vraag 2 — Cluster-IP, DNS, EndpointSlices

Antwoord. Het **cluster-IP** (`10.43.x.x`) is een virtueel adres dat de API server toekent bij het aanmaken van het Service, stabiel zolang het bestaat. De **DNS-naam** `api.lab13.svc.cluster.local` wordt bijgehouden door CoreDNS, dat de Services observeert. De **EndpointSlices** zijn de lijst van `IP:poort` van de Pods die het label van de selector dragen **en** klaar zijn, bijgehouden door de endpoints-controller. `ping` faalt omdat het IP op geen enkele interface bestaat: de regels van kube-proxy vertalen alleen TCP/UDP-verbindingen naar een poort van het Service, geen ICMP.

Waarom. Het Service is een abstractie van gedistribueerde NAT, geen machine.

Nuance. Met kube-proxy in IPVS-modus (of Cilium zonder kube-proxy) kan het IP wél op `ping` antwoorden — vertrouw niet op `ping` om een Service te diagnosticeren.

Voorbeeld.

```
kubectl get svc api -o jsonpath='{.spec.clusterIP}' ; kubectl get endpointslices -l kubernetes.io/service-name=api
```

Vraag 3 — De endpoints volgen de Pods

Antwoord. Na de overgang naar 3 replica's: **drie** adressen (zodra de derde Pod klaar is). Na het verwijderen van een Pod: het adres verdwijnt **onmiddellijk**, en dat van de vervangende Pod verschijnt wanneer die klaar is (enkele seconden met twee adressen). Na de gebroken readiness: het adres van die Pod wordt **weggehaald**, er blijven er twee, de Pod blijft draaien. De client die

`api:8080` aanriep, heeft niets gemerkt, behalve een afgebroken verbinding als hij precies op de verwijderde Pod zat op het verkeerde moment.

Waarom. De endpoints-controller reageert op wijzigingen van Pods (aanmaak, verwijdering, conditie `Ready`) en kube-proxy werkt zijn regels in minder dan een seconde bij.

Nuance. Tussen het moment waarop een Pod `SIGTERM` krijgt en zijn verwijdering uit de endpoints kunnen hem nog enkele verzoeken bereiken: daarom moet een applicatie het nette afsluiten beheersen (lab 03) en voegt men soms een `preStop` van enkele seconden toe.

Voorbeeld.

```
kubectl get endpointslices -l kubernetes.io/service-name=api -o
jsonpath='{.items[0].endpoints[*].addresses[0]}' ; echo
```

Vraag 4 — Een Service zonder endpoints

Antwoord. Ofwel komt de **selector** met geen enkel Pod-label overeen (`app: apis`, `app: backend`, een tikfout), ofwel zijn de Pods **niet klaar** — maar hier zijn ze `1/1`, dus is het de selector. `kubectl get pods --show-labels` vergelijken met `kubectl get svc api -o jsonpath='{.spec.selector}'` geeft uitsluitsel. Correctie: de selector (of de labels van het template) verbeteren en opnieuw toepassen; voor het readiness-geval, de sonde verbeteren.

Waarom. De EndpointSlice wordt berekend uit de selector; geen overeenkomst, geen adres, en Traefik, dat naar de endpoints routeert, heeft niets om naar te routeren: `503`.

Nuance. `kubectl describe svc api` toont `Endpoints: <none>` in klare taal; dat is de regel om te zoeken.

Voorbeeld.

```
kubectl get svc api -o jsonpath='{.spec.selector}' ; echo ; kubectl get pods --show-labels |
head -3
```

Vraag 5 — Drie manieren om de frontend bloot te stellen

Antwoord. ClusterIP: geen URL vanaf de host, alleen `kubectl port-forward svc/web 18080:80` en dan `http://localhost:18080`. NodePort: `http://<ip-van-de-node>:30080` (en vaak `http://localhost:30080` vanuit WSL). LoadBalancer: `http://<EXTERNAL-IP>:8081`, waarbij het `EXTERNAL-IP` het IP van de node is omdat **ServiceLB** van k3s een Pod `svclb-web-lb` start die de poort op de node opent — kale Kubernetes zou `<pending>` laten bij gebrek aan leverancier. Een tweede LoadBalancer op poort 80 zou `<pending>` blijven omdat ServiceLB dezelfde poort niet twee keer op dezelfde node kan openen, en Traefik ze al bezet.

Waarom. ServiceLB is een "load balancer voor armen": één hostpoort per Service, op elke node.

Nuance. In de cloud bestelt dezelfde YAML een echte load balancer met een eigen IP; de poorten botsen niet meer, maar elk kost geld.

Voorbeeld.

```
kubectl get pods -n kube-system -l svccontroller.k3s.cattle.io/svcname=web-lb -o name   #
pod/svclb-web-lb-…
```

Vraag 6 — De korte naam werkt alleen in de namespace

Antwoord. De `/etc/resolv.conf` van een Pod bevat `search lab13.svc.cluster.local svc.cluster.local cluster.local` en `nameserver 10.43.0.10` (CoreDNS). Vanuit `lab13` wordt `api` via het eerste achtervoegsel `api.lab13.svc.cluster.local`. Vanuit `default` is het eerste achtervoegsel `default.svc.cluster.local`: `api.default.svc.cluster.local` bestaat niet, en `api.svc.cluster.local` evenmin. Je moet `api.lab13` schrijven (het tweede achtervoegsel vult aan tot `api.lab13.svc.cluster.local`) of de volledige naam.

Waarom. De `search` wordt door de kubelet gegenereerd met de namespace van de Pod; `ndots:5` laat de achtervoegsels proberen vóór een rechtstreekse resolutie.

Nuance. Een Service van een andere namespace aanroepen is standaard toegestaan; een NetworkPolicy (lab 16) kan het verbieden.

Voorbeeld.

```
kubectl exec deploy/web -- cat /etc/resolv.conf
kubectl run -n default -it --rm t --image=docker.io/library/alpine -- wget -qO-
http://api.lab13:8080/actuator/health
```

Vraag 7 — Verbindingen, geen verzoeken

Antwoord. kube-proxy kiest een Pod **bij het openen van elke TCP-verbinding** en kijkt nooit naar de inhoud. Een HTTP/1.1-client met *keep-alive* (of een JDBC-pool) opent weinig verbindingen en houdt ze: al zijn verzoeken gaan naar **dezelfde** Pod, en drie replica's delen de belasting alleen als er genoeg clients zijn. De Ingress (Traefik) beëindigt de verbinding van de client en opent zijn eigen verbindingen naar de Pods: hij verdeelt **per HTTP-verzoek**, en kan zelfs *sticky* of gewogen werken.

Waarom. Het Service is een mechanisme van laag 4; load balancing op laag 7 vraagt een proxy die HTTP begrijpt.

Nuance. Voor een database (lange verbindingen) is dat aanvaardbaar en zelfs gewenst; voor een API die door één frontend aangeroepen wordt, is het het argument om via de Ingress te gaan in plaats van via de nginx-proxy in de `web`-Pod.

Voorbeeld.

```
for i in 1 2 3 4 ; do kubectl exec deploy/web -- wget -qO- http://api:8080/ ; done   # elke
wget = nieuwe verbinding = willekeurige Pod
```

Vraag 8 — `port` en `targetPort`

Antwoord. `port: 80` is de poort **van het Service** (die je aanroept: `api:80`, dus `http://api/`); `targetPort: 8080` is de poort **van de container** waarnaar kube-proxy vertaalt. Op `api:8080` bedient niemand, vandaar `Connection refused`. nginx moet `proxy_pass http://api/;` schrijven (of `http://api:80/`).

Waarom. Het Service ontkoppelt de blootgestelde poort van de echte poort, wat toelaat de ene te veranderen zonder de andere — en alle Services een "nette" poort te geven.

Nuance. `targetPort` kan een **naam** van een containerpoort zijn (`targetPort: http` als de container `name: http` declareert), wat een nummerwijziging in de image overleeft.

Voorbeeld.

```
kubectl get svc api -o jsonpath='{.spec.ports[0].port} -> {.spec.ports[0].targetPort}' ; echo
```

Vraag 9 — Het pad van een verzoek

Antwoord. Windows-browser → WSL-forwarding → poort **80** van de node, geopend door de Pod `svclb-traefik` (ServiceLB) → Service `traefik` → **Traefik**-Pod, die de Ingress `web` leest (`/` → Service `web`) → Service `web` → Pod `web` (**nginx**), waarvan de configuratie `/api/` doorstuurt → Service `api` → een `api`-Pod, die `{"status":"UP"}` antwoordt. Twee reverse proxies: Traefik en dan nginx. Vereenvoudiging: aan de Ingress een regel `path: /api` → Service `api` poort 8080 toevoegen (met herschrijving van het voorvoegsel), en de `location /api/` uit nginx halen — Traefik routeert rechtstreeks.

Waarom. Elk onderdeel kent alleen het volgende: dat maakt elke schakel vervangbaar.

Nuance. De herschrijving `/api/x` → `/x` gebeurt bij Traefik met een *middleware* (`stripPrefix`), bij ingress-nginx met een annotatie `rewrite-target` : de syntaxis hangt af van de controller, niet van het standaard Ingress-object.

Voorbeeld.

```
kubectl describe ingress web | grep -A4 Rules
```

Vraag 10 — Ingress en Ingress controller

Antwoord. De `Ingress` is een object van de API: routeringsregels, meer niet. De *Ingress controller* is een **programma** (Traefik, nginx…) dat in het cluster draait, die objecten observeert en zijn reverse proxy ernaar configureert. Zonder controller wordt een Ingress aanvaard, opgeslagen, en is hij **zonder effect**: geen `ADDRESS` , geen verkeer. Een `503` van Traefik betekent dat Traefik het verzoek wel ontvangen heeft, de regel gevonden heeft, maar dat het beoogde Service **geen enkel klaar endpoint** heeft: het probleem zit stroomafwaarts (Pods, readiness, selector), Traefik is maar de boodschapper.

Waarom. Kubernetes definieert de interface, niet de implementatie; hetzelfde model als CNI (netwerk) en CSI (opslag).

Nuance. Een cluster kan meerdere controllers hebben; `spec.ingressClassName` (of de standaardklasse, `traefik` bij k3s) zegt welke het object neemt.

Voorbeeld.

```
kubectl get ingressclass ; kubectl get ingress web -o jsonpath='{.spec.ingressClassName}'
```

Vraag 11 — De Ingress zonder `ADDRESS` op EKS

Antwoord. Er ontbreekt een **Ingress controller**: k3s levert er een (Traefik), EKS niet. De Ingress-objecten worden opgeslagen maar niemand leest ze; het `ADDRESS` blijft leeg. Je moet een controller installeren (ingress-nginx, Traefik, of de AWS Load Balancer Controller) en, naargelang het geval, `ingressClassName` invullen. Het "werkte" op k3s omdat Traefik er al stond, met zijn standaardklasse.

Waarom. Distributies kiezen wat ze meeleveren; een beheerd cluster laat die keuze aan het platformteam.

Nuance. Op EKS werkt het `LoadBalancer`-Service zonder iets te installeren omdat de cloudintegratie **wel** geleverd wordt (de cloud-controller-manager): het omgekeerde van k3s, waar ServiceLB simuleert.

Voorbeeld.

```
kubectl get pods -A | grep -i -E 'ingress|traefik'   # leeg = geen controller
```

Vraag 12 — `/api/` via nginx of via de Ingress

Antwoord. Zelfde origin = geen CORS, één enkele URL om in te stellen, de frontend weet niet waar de API woont (lab 07). In onze stack kan de `/api/`-routering gebeuren door **nginx** (de `location /api/` van de `web`-Pod) of door de **Ingress** (een regel `path: /api`). Aan de Ingress toevertrouwen: één proxy in plaats van twee, de image `web` hoeft de naam `api` niet meer te kennen (dus geen crash meer bij het opstarten), de verdeling per verzoek van Traefik over de `api`-Pods, TLS en headers op één plaats, en een frontend herleid tot statische bestanden.

Waarom. Routering is een verantwoordelijkheid van het platform; de statische frontend hoeft ze niet te dragen.

Nuance. De proxy in nginx blijft nuttig op het werkstation (Compose, lab 09) waar geen Ingress is: sommige teams houden beide configuraties.

Voorbeeld.

```
- path: /api
  pathType: Prefix
  backend: { service: { name: api, port: { number: 8080 } } }
```

Vraag 13 — De database bereiken vanuit Windows

Antwoord. Toenemende blootstelling: `port-forward` (een versleutelde tunnel via de API server, geldig zolang het commando loopt, alleen voor jou zichtbaar) < NodePort (een poort open op alle nodes, voor het hele netwerk) < LoadBalancer (een eigen IP, publiek in de cloud). In productie is alleen de eerste aanvaardbaar, en dan nog, voor een eenmalige diagnose met de juiste RBAC-rechten (lab 16): een database heeft nooit een poort naar buiten. Wat nog ontbreekt: een **Service** `db` (ClusterIP, of *headless* voor een StatefulSet, lab 15) opdat `db` een naam heeft in het cluster.

Waarom. Elk type Service vergroot het blootstellingsoppervlak; de behoefte van een mens is niet die van de applicatie.

Nuance. In het bedrijf loopt de menselijke toegang tot databases via een bastion of een audittool (Teleport, Boundary), nooit via een nodepoort.

Voorbeeld.

```
kubectl port-forward svc/db 5432:5432   # dan DBeaver op localhost:5432, zolang de sessie duurt
```